



UNIVERZITET DONJA GORICA

FPRN – ELEKTROTEHNIKA I RAČUNARSTVO

Kompjuterske mreže i telekomunikacije

**DOMAĆI ZADATAK 2
TCP I UDP KOMUNIKACIJA**

Profesor: Prof. dr. Nenad Krajnović

Ime i prezime: Leona Markić

prof.dr. Tomo Popović

Indeks: 23/038

Datum: maj, 2026.godine

Sadržaj:

Tabela slika.....	3
Uvod	4
1. TCP komunikacija.....	5
2. UDP komunikacija	7
3. Wireshark analiza.....	9
AI dnevnik	12
Zaključak	12

Tabela slika

Slika 1 Kod korišten za implementaciju TCP servera i klienta	6
Slika 2 Razmjena poruka između TCP klijenta i servera.....	6
Slika 3 Kod korišten za implementaciju UDP receiver i sender aplikacije	8
Slika 4 Razmjena 100 poruka između UDP sender i receiver aplikacije.....	8
Slika 5 Analiza TCP komunikacije u Wireshark alatu	9
Slika 6 Prikaz sadržaja TCP paketa u Wireshark alatu	10
Slika 7 Prikaz UDP paketa i sadržaja poruke u Wireshark alatu	11
Slika 8 Prikaz sadržaja UDP paketa u Wireshark alatu.....	11

Uvod

Cilj ovog zadatka je implementacija mrežne komunikacije korištenjem TCP i UDP protokola u Python programskom jeziku. Zadatak obuhvata razvoj jednostavnih aplikacija koje komuniciraju putem mreže, kao i analizu razmjene podataka pomoću Wireshark alata. TCP (Transmission Control Protocol) je pouzdan protokol koji osigurava isporuku podataka bez grešaka i u ispravnom redoslijedu. Prije same komunikacije uspostavlja se konekcija između klijenta i servera. UDP (User Datagram Protocol) je jednostavniji i brži protokol koji ne uspostavlja konekciju i ne garantuje isporuku podataka niti njihov redoslijed. U okviru zadatka implementirane su obje vrste komunikacije, izvršen je eksperiment sa slanjem većeg broja poruka, te je analiziran mrežni saobraćaj korištenjem Wiresharka.

Tokom realizacije zadatka implementirana je i interaktivna chat komunikacija korištenjem while petlje, gdje je korisnik ručno unosio poruke radi provjere uspješnog prenosa podataka i očuvanja redoslijeda poruka između aplikacija.

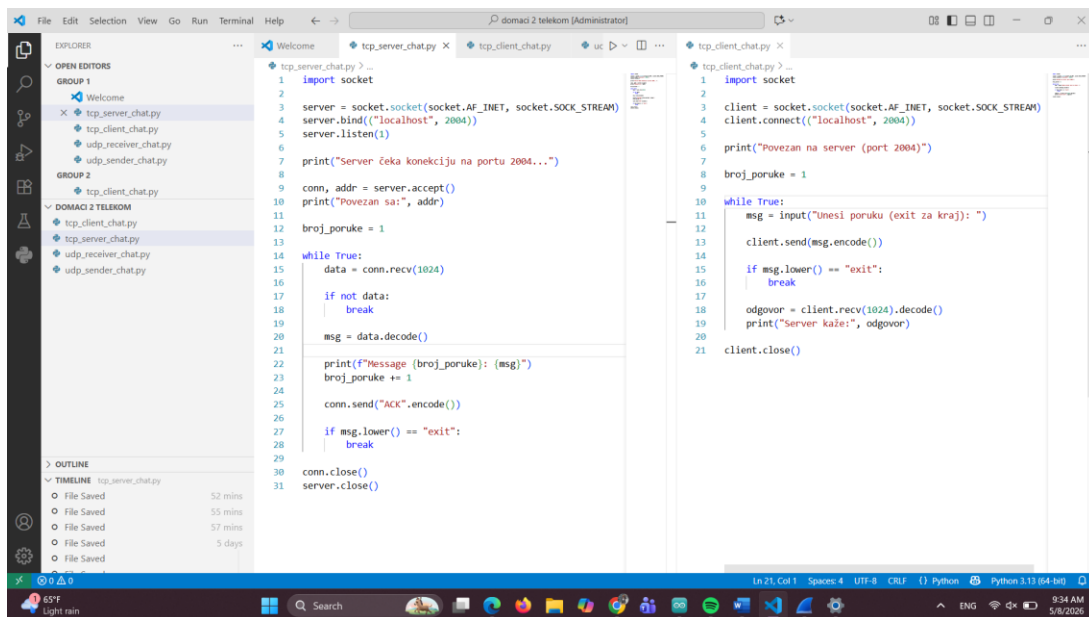
1. TCP komunikacija

Za realizaciju TCP komunikacije razvijene su dvije aplikacije: server i klijent. Server sluša dolazne konekcije na određenom portu, dok klijent uspostavlja vezu sa serverom i šalje poruke. TCP protokol je konekcioni protokol, što znači da se prije razmjene podataka uspostavlja veza između klijenta i servera. Ovaj proces se naziva "three-way handshake" i sastoji se od tri koraka: SYN, SYN-ACK i ACK. U implementaciji, server koristi socket koji je vezan za određeni port i čeka zahtjev za konekcijom. Nakon uspostavljanja veze, server prima poruke od klijenta i šalje potvrdu prijema (ACK).

U nastavku je prikazan dio implementacije TCP komunikacije u Python programskom jeziku. U implementaciji je korišten Python programski jezik i socket biblioteka. Server kreira TCP socket korištenjem `socket.AF_INET` i `socket.SOCK_STREAM`. Nakon toga server se vezuje za lokalnu adresu (localhost) i port 2004 pomoću funkcije `bind()`. Funkcija `listen()` omogućava serveru da osluškuje dolazne konekcije, dok funkcija `accept()` prihvata zahtjev za povezivanje od strane klijenta.

Komunikacija između klijenta i servera realizovana je pomoću while petlje. Klijent šalje poruke serveru korištenjem funkcije `send()`, dok server prima podatke pomoću funkcije `recv()`. Nakon prijema poruke server šalje aplikacionu potvrdu prijema u obliku tekstualne poruke „ACK“.

Kada korisnik unese poruku „exit“, komunikacija se prekida i konekcija se zatvara korištenjem funkcije `close()`.

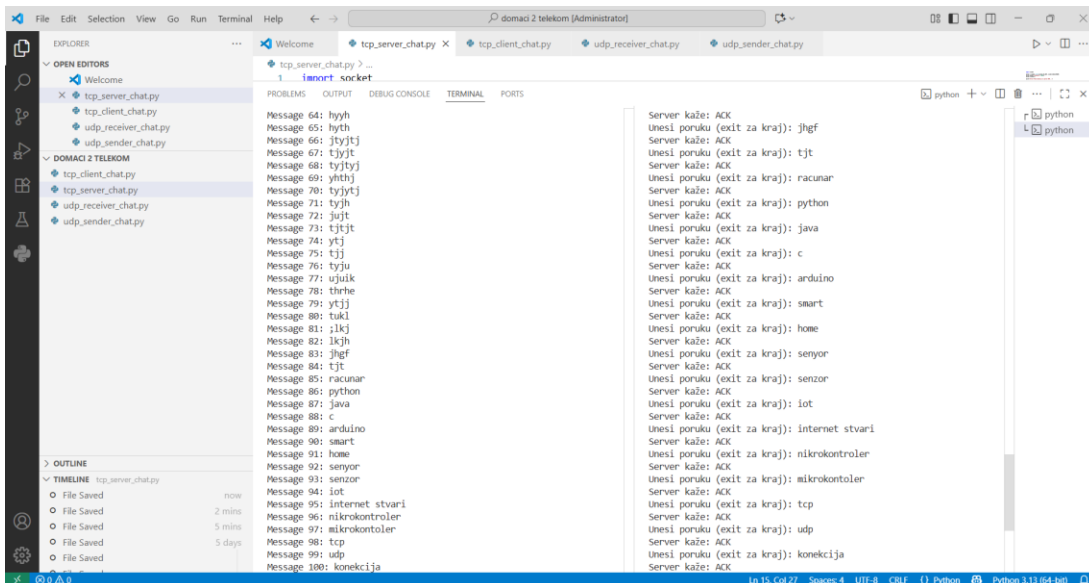


Slika 1 Kod korišten za implementaciju TCP servera i klienta

Za pokretanje TCP aplikacija korišten je terminal u razvojnom okruženju Visual Studio Code.

Prvo je pokrenut TCP server komandom: `python tcp_server_chat.py`

Nakon toga je u drugom terminalu pokrenut TCP klijent komandom: `python tcp_client_chat.py`



Slika 2 Razmjena poruka između TCP klijenta i servera

Na ovaj način ostvarena je komunikacija između klijenta i servera, pri čemu je klijent slao poruke, a server ih primao i slao potvrdu prijema.

Tokom testiranja izvršen je eksperiment sa slanjem 100 poruka između TCP klijenta i servera korištenjem while petlje. Na osnovu rezultata uočeno je da su sve poruke uspješno primljene i prikazane u ispravnom redoslijedu, što potvrđuje pouzdanost TCP protokola.

2. UDP komunikacija

Za realizaciju UDP komunikacije razvijene su dvije aplikacije: UDP sender i UDP receiver. Za razliku od TCP protokola, UDP (User Datagram Protocol) predstavlja nekonekcionu protokol, što znači da se prije slanja podataka ne uspostavlja konekcija između aplikacija.

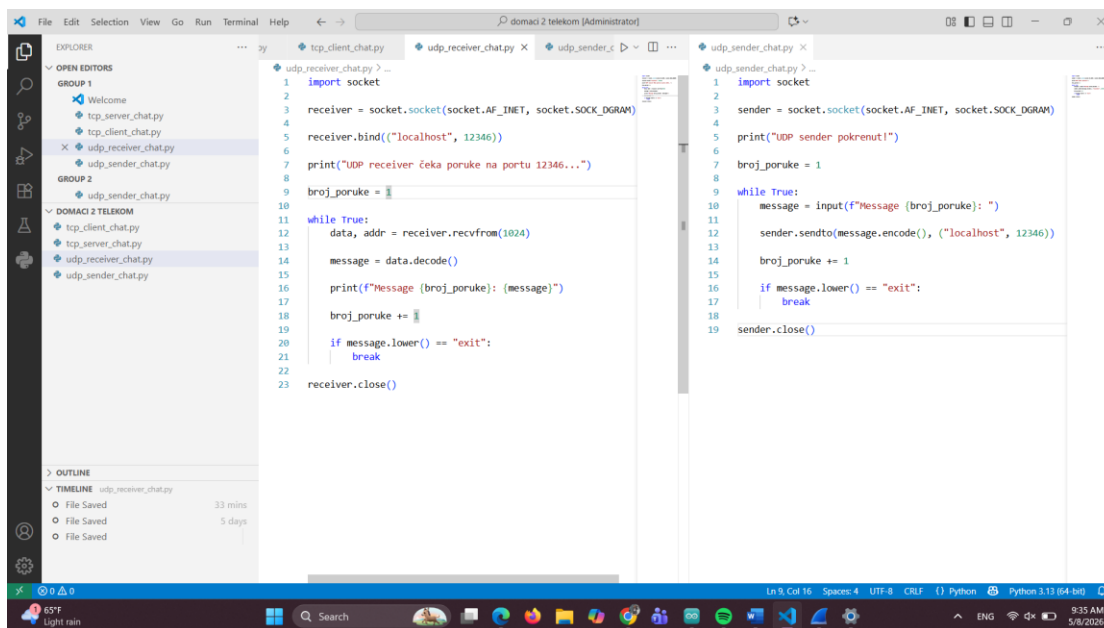
UDP omogućava jednostavniji i brži prenos podataka, ali ne garantuje pouzdanost komunikacije, isporuku podataka niti očuvanje redoslijeda poruka. Podaci se šalju direktno u obliku datagrama bez mehanizma potvrde prijema.

U implementaciji je korišten Python programski jezik i socket biblioteka. Receiver aplikacija kreira UDP socket korištenjem `socket.AF_INET` i `socket.SOCK_DGRAM`. Nakon toga receiver se vezuje za lokalnu adresu (localhost) i port 12346 korištenjem funkcije `bind()`.

Receiver kontinuirano osluškuje dolazne poruke pomoću while petlje i funkcije `recvfrom()`. Primljene poruke se dekodiraju i prikazuju u terminalu.

Sender aplikacija šalje poruke receiver aplikaciji korištenjem funkcije `sendto()`. Korisnik unosi poruke putem tastature, nakon čega se poruke šalju preko UDP protokola.

Kada korisnik unese poruku „exit“, komunikacija se prekida i socket se zatvara korištenjem funkcije `close()`.

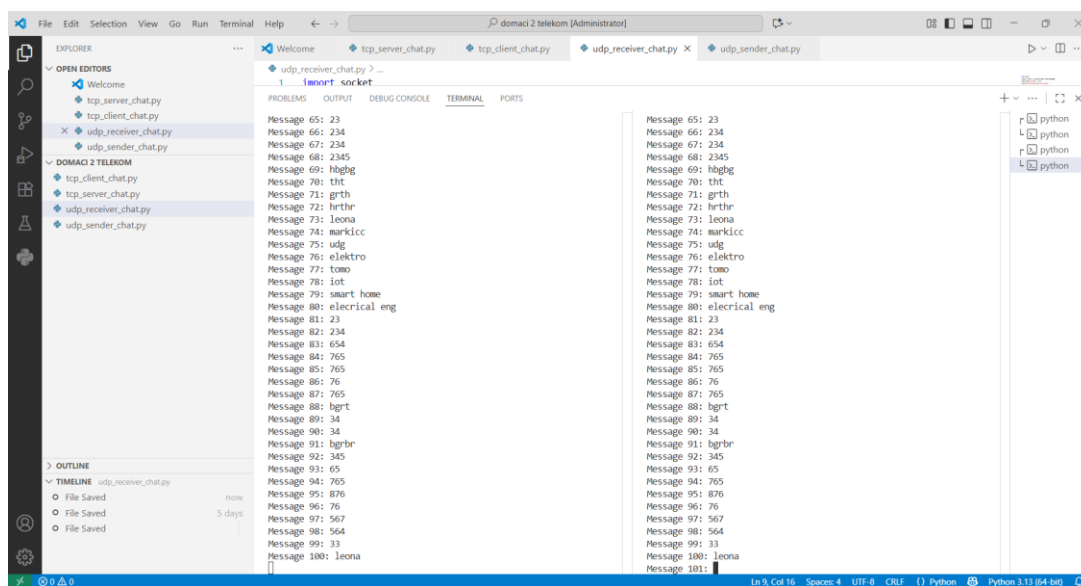


Slika 3 Kod korišten za implementaciju UDP receiver i sender aplikacije

Za pokretanje UDP aplikacija korišten je terminal u razvojnom okruženju Visual Studio Code.

Prvo je pokrenut UDP receiver komandom: `python udp_receiver_chat.py`

Nakon toga je u drugom terminalu pokrenut UDP sender komandom: `python udp_sender_chat.py`



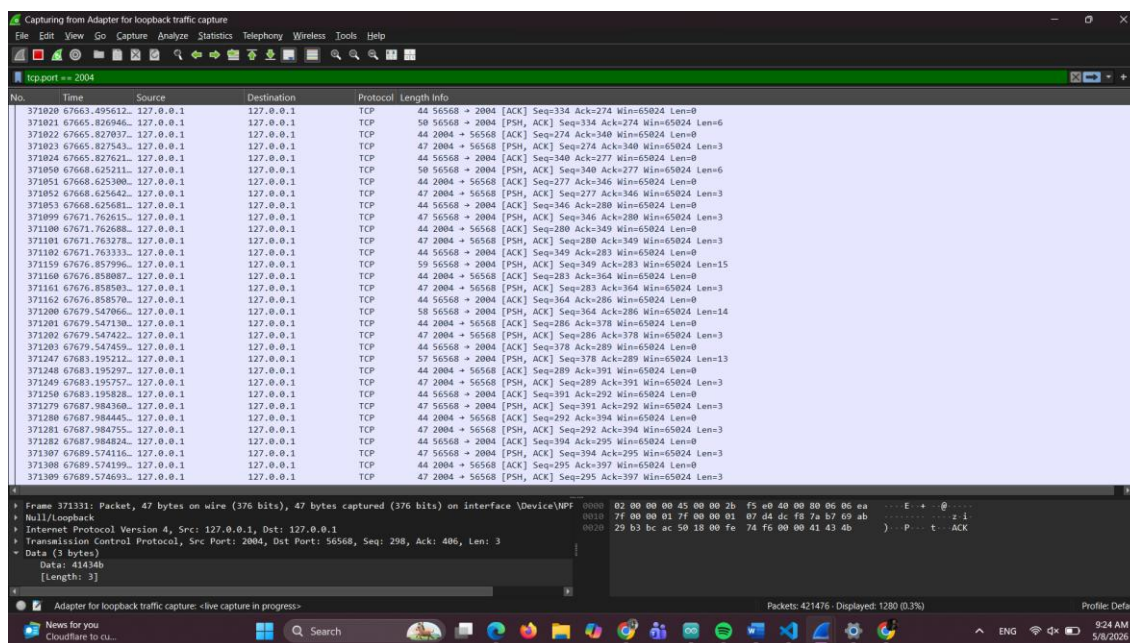
Slika 4 Razmjena 100 poruka između UDP sender i receiver aplikacije

Na ovaj način ostvarena je komunikacija između sender i receiver aplikacije korištenjem UDP protokola.

Tokom testiranja izvršen je eksperiment sa slanjem 100 poruka između UDP sender i receiver aplikacije korištenjem while petlje. U toku komunikacije na localhost mreži nije uočen gubitak poruka niti promjena redoslijeda, iako UDP protokol u realnim mrežnim uslovima ne garantuje pouzdanost prenosa podataka.

3. Wireshark analiza

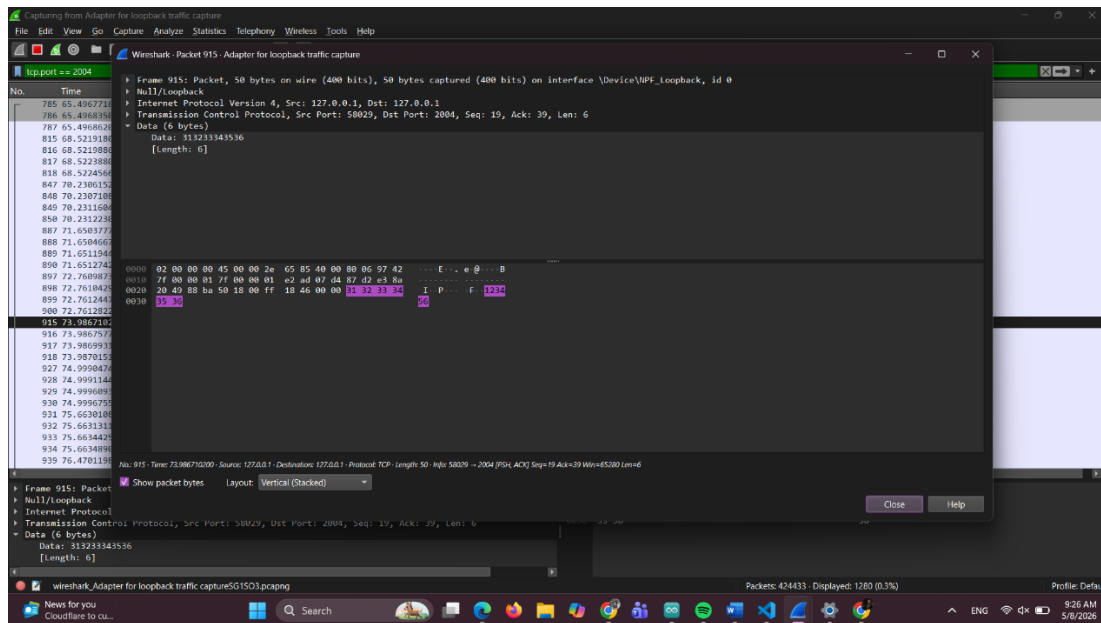
Za analizu mrežnog saobraćaja korišten je Wireshark alat. Snimanje mrežnih paketa izvršeno je na lokalnom loopback interfejsu, jer su klijent i server komunicirali putem lokalne adrese 127.0.0.1 (localhost). Za analizu TCP komunikacije korišten je filter: tcp.port == 2004



Slika 5 Analiza TCP komunikacije u Wireshark alatu

Na slici 5 je prikazana analiza TCP komunikacije korištenjem Wireshark alata. Za filtriranje TCP saobraćaja korišten je filter tcp.port == 2004. Uočeni su TCP paketi sa oznakama PSH i ACK, koji predstavljaju razmjenu podataka između klijenta i servera, kao i potvrdu prijema podataka. U Data dijelu paketa prikazana je aplikaciona potvrda „ACK“ koju server šalje klijentu nakon prijema

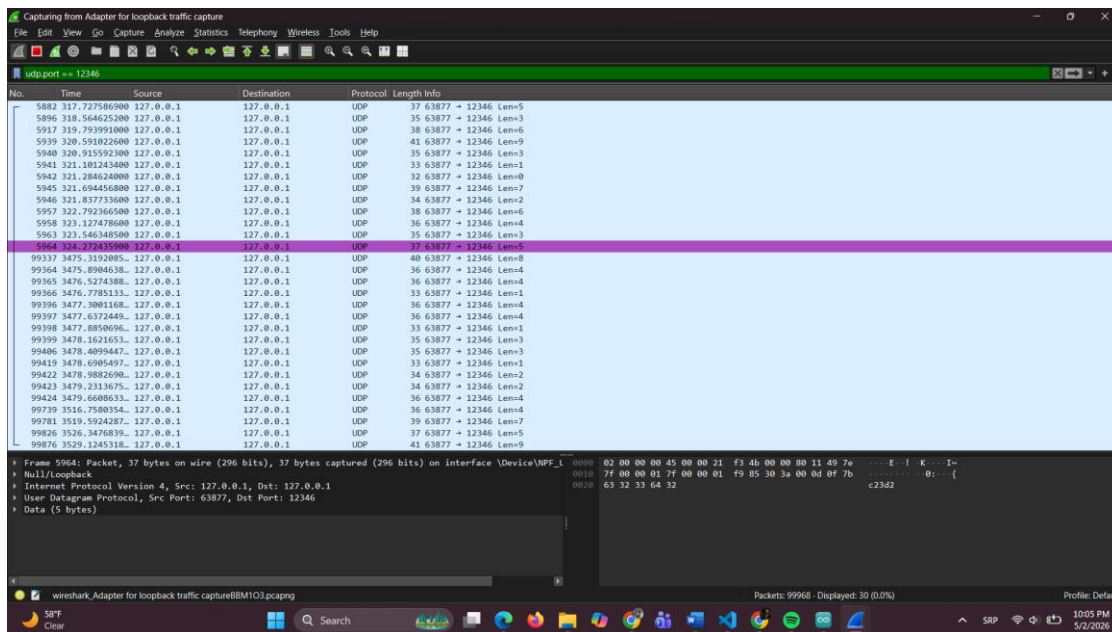
poruke. Takođe su vidljivi izvorni i odredišni portovi, kao i Sequence i Acknowledgment vrijednosti koje TCP koristi za pouzdanu komunikaciju i očuvanje redosljeda podataka.



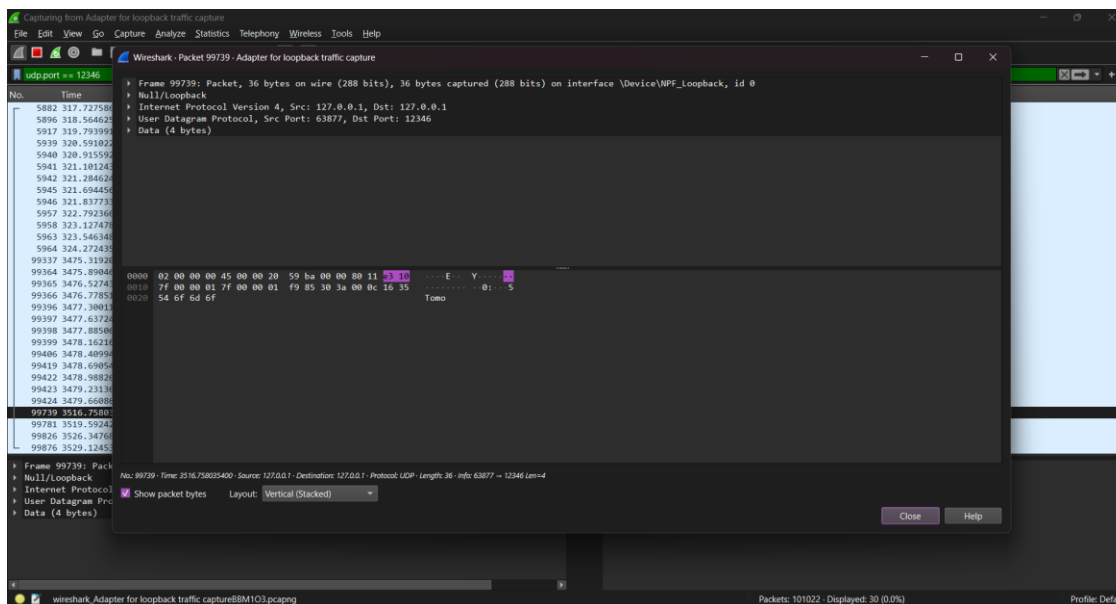
Slika 6 Prikaz sadržaja TCP paketa u Wireshark alatu

Na slici 6 je prikazan sadržaj TCP paketa analiziran pomoću Wireshark alata. U Data dijelu paketa vidljiva je korisnička poruka „123456“ koja je uspješno poslata od strane TCP klijenta i primljena na serveru. Ovaj prikaz potvrđuje uspješnu razmjenu podataka između klijenta i servera korištenjem TCP protokola.

Takođe su prikazani izvorni i odredišni portovi, kao i TCP oznake PSH i ACK, koje označavaju prenos podataka i potvrdu prijema paketa.



Slika 7 Prikaz UDP paketa i sadržaja poruke u Wireshark alatu



Slika 8 Prikaz sadržaja UDP paketa u Wireshark alatu

U sadržaju UDP paketa (Data dio) vidljive su tekstualne poruke koje su poslate između sender i receiver aplikacije. Na slici se može vidjeti primjer korisničke poruke unutar UDP paketa, što potvrđuje uspješnu razmjenu podataka putem UDP protokola.

Na osnovu izvršene analize može se zaključiti da TCP protokol omogućava pouzdanu komunikaciju korištenjem uspostavljanja konekcije i potvrde prijema podataka, dok UDP omogućava jednostavniji i brži prenos bez mehanizama kontrole i garancije isporuke.

Tokom eksperimenta izvršeno je automatsko slanje 100 poruka korištenjem TCP i UDP protokola.

AI dnevnik

Tokom izrade domaćeg zadatka korišten je AI alat ChatGPT kao pomoć pri: objašnjenju TCP i UDP komunikacije, organizaciji i pisanju izvještaja, provjeri gramatičkih i tehničkih grešaka, objašnjenju rada Wireshark alata.

Za implementaciju i testiranje aplikacija korišteno je razvojno okruženje Visual Studio Code, kao i Wireshark alat za analizu mrežnog saobraćaja.

Sav kod je testiran i prilagođen od strane autora rada.

Zaključak

U okviru ovog zadatka uspješno je implementirana komunikacija između dvije aplikacije korištenjem TCP i UDP protokola u Python programskom jeziku. Kroz praktičan rad omogućeno je bolje razumijevanje načina prenosa podataka u računarskim mrežama.

TCP protokol se pokazao kao pouzdan način komunikacije, gdje su sve poruke uspješno prenesene i primljene u ispravnom redoslijedu. Uočeno je da TCP koristi dodatne mehanizme kontrole, kao što su uspostavljanje konekcije i potvrda prijema, što osigurava tačnost prenosa podataka.

Sa druge strane, UDP protokol omogućava jednostavniji i brži prenos podataka, ali bez garancije isporuke i redoslijeda. Iako su u ovom eksperimentu poruke uspješno razmijenjene, u realnim uslovima može doći do gubitka podataka.